

Explicação do projeto NEABI

Guia das alterações feitas em `TelaPrincipal.html`, `Estilo.css` e `script.js`.

Documento gerado em 06/10/2026.

1. Como os arquivos trabalham juntos

`TelaPrincipal.html` define o conteúdo e a estrutura: menu, títulos, textos, imagens e links.

`Estilo.css` define a aparência: cores, tamanhos, espaçamentos, Grid, Flexbox, responsividade e animações.

`script.js` adiciona comportamento: aumenta links quando o mouse passa por cima e revela elementos conforme a pessoa rola a página.

Regra geral: HTML é a estrutura, CSS é a apresentação e JavaScript é a interação.

2. HTML: estrutura inicial

Declaração e cabeçalho

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="Estilo.css">
</head>
```

- `<!DOCTYPE html>` informa que o documento usa HTML5.
- `lang="en"` declara o idioma do documento; como o conteúdo é português, o ideal futuro é trocar para `pt-BR`.
- `meta charset` permite acentos e caracteres especiais.
- `meta viewport` faz o layout se adaptar à largura do celular.
- `title` define o texto da aba do navegador.
- `link` conecta o HTML ao arquivo de estilos.

Menu de navegação

```
<nav>
  <a href="#nossa-historia">Nossa História</a>
  <a href="#gaspar">Em Gaspar</a>
  <a href="#Herculano">Coordenador</a>
  <a href="#oportunidades">Oportunidades</a>
  <a href="#atividades">Ações</a>
</nav>
```

- `nav` identifica a área de navegação.
- Cada `a` é um link.
- O valor iniciado por `#` aponta para um elemento que possui o mesmo `id`.
- Assim, clicar em "Oportunidades" leva ao elemento `id="oportunidades"`.

Imagem principal e seção Nossa Historia

```

<div id="NSH_di">
  <h1 id="nossa-historia">Nossa Historia</h1>
  <div class="conteudo-paragrafo01">
    <p id="paragrafo01">...</p>
    
  </div>
</div>
```

- `id="principal"` permite controlar a imagem inicial no CSS.
- `src` indica o arquivo da imagem.
- `alt` descreve a imagem para acessibilidade e quando ela não carrega.
- `conteudo-paragrafo01` agrupa texto e imagem para colocá-los lado a lado.
- O `id` deve ser único; a classe pode ser reutilizada.

Seção do idealizador

```
<div id="Herculano">
  <h1>PRINCIPAL IDEALIZADOR| COORDENADOR</h1>
  <div class="conteudo-herculano">
    <p>Luiz Herculano de Sousa Guilherme ...</p>
    
  </div>
</div>
```

- `id="Herculano"` identifica a seção e recebe o link do menu.
- `conteudo-herculano` organiza texto e imagem em uma linha.
- `flex-direction: row-reverse` no CSS coloca a imagem no lado oposto ao texto.

Oportunidades

```
<section class="oportunidades">
  <h2 id="oportunidades">OPORTUNIDADES</h2>
  <p>...</p>
  <p>...</p>
  <p>...</p>
  <p>...</p>
</section>
```

- `section` cria uma área semântica própria.
- A classe permite aplicar a imagem de fundo somente nessa área.
- Todos os parágrafos dentro dela recebem o mesmo fundo visual.

Links finais com imagens

```
<div class="links-finais">
  <a class="imagem-link-final imagem-link-abpn"
    href="https://abpn.org.br/" aria-label="Acessar o site da ABPN">
    
  </a>
  <a class="imagem-link-final imagem-link-copene"
    href="https://www.copene2026.abpn.org.br/"
    aria-label="Acessar o site do COPENE 2026">
    
  </a>
</div>
```

- Colocar a imagem dentro do `a` torna a imagem inteira clicável.
- `href` define o destino.
- `aria-label` explica a ação para leitores de tela.
- As duas imagens usam classes específicas, mas compartilham a mesma regra de tamanho.

3. CSS: aparência e layout

Variáveis e corpo da página

```
:root {
  background-color: #ec9039;
  padding: 80px;
}

body {
  margin: 0;
  position: relative;
  min-height: 100vh;
}

body {
  font-family: Arial, Helvetica, sans-serif;
  color: #2b1a0c;
}
```

- `:root` representa a raiz do documento e recebe o fundo laranja e o espaçamento externo.
- `margin: 0` remove a margem padrão do navegador.
- `position: relative` cria uma referência de posicionamento.
- `min-height: 100vh` garante pelo menos a altura da janela.
- `font-family` define a família da fonte.
- `color` define a cor padrão do texto.

Imagem principal e imagens gerais

```
#principal {
  display: block;
  max-width: 15%;
  height: auto;
  margin: 25px auto;
}

img {
  max-width: 100%;
  height: auto;
}

#imagem-historia {
  width: 320px;
  border-radius: 8px;
}
```

- `display: block` permite centralizar a imagem com margem automática.
- `max-width: 15%` limita a imagem principal no desktop.
- `height: auto` conserva a proporção.
- `margin: 25px auto` cria espaço vertical e centraliza horizontalmente.
- `max-width: 100%` impede imagens de ultrapassarem o contêiner.
- `border-radius: 8px` arredonda os cantos.

Flexbox das seções com texto e imagem

```
.conteudo-paragrafo01 {
  display: flex;
  align-items: flex-start;
  gap: 30px;
  max-width: 1200px;
}

.conteudo-herculano {
  display: flex;
  flex-direction: row-reverse;
  align-items: flex-start;
  gap: 30px;
  max-width: 1200px;
}
```

- `display: flex` organiza filhos em uma linha.
- `align-items: flex-start` alinha os itens pelo topo.
- `gap: 30px` cria espaço entre texto e imagem.
- `max-width` evita linhas excessivamente largas.
- `row-reverse` inverte a ordem visual no bloco do idealizador.

Espaçamento e textos

```
#gaspar { margin-top: 60px; }

#paragrafo01 {
  width: 800px;
  margin: 0;
  font-size: 22px;
  line-height: 1.5;
  text-align: left;
}

#paragrafo02 { font-size: 20px; line-height: 1.5; }
#Herculano p { font-size: 18px; line-height: 1.6; }
```

- `margin-top: 60px` aumenta a distância entre Nossa Historia e Gaspar.
- `width` define a largura do primeiro texto.
- `margin: 0` remove espaço padrão do parágrafo.
- `font-size` controla o tamanho das letras.
- `line-height` controla a distância entre linhas e melhora a leitura.
- `text-align: left` alinha o texto à esquerda.

Fundo da seção Oportunidades

```
.oportunidades {
  position: relative;
  overflow: hidden;
  padding: 30px;
  background-image: linear-gradient(
    rgba(236, 144, 57, 0.82),
    rgba(236, 144, 57, 0.82)
  ), url("apresentando copene.jpeg");
  background-size: 140% auto;
  background-position: center;
  background-repeat: no-repeat;
}
```

- `position: relative` prepara a seção para posicionamentos internos.
- `overflow: hidden` esconde partes da imagem que ultrapassariam a área.
- `padding: 30px` separa o texto das bordas.

- `linear-gradient` cria uma camada laranja semitransparente sobre a foto.
- `rgba(..., 0.82)` usa 82% de opacidade; por isso a imagem fica quase apagada.
- `background-size: 140% auto` amplia a imagem e provoca o corte desejado.
- `background-position: center` mantém o centro da foto visível.
- `background-repeat: no-repeat` impede repetição.

Grid dos links finais

```
.links-finais {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  align-items: center;
  gap: 20px;
}

.imagem-abpn,
.imagem-copene {
  display: block;
  width: 100%;
  height: 240px;
  object-fit: cover;
  border-radius: 8px;
}
```

- `display: grid` ativa CSS Grid.
- `repeat(2, 1fr)` cria duas colunas de mesma proporção.
- `align-items: center` alinha os itens verticalmente.
- `gap: 20px` separa as colunas.
- `width: 100%` faz cada imagem preencher sua coluna.
- `height: 240px` deixa as duas com a mesma altura.
- `object-fit: cover` corta a imagem sem deformá-la.

Menu fixo e efeito de aumento

```
nav {
  position: sticky;
  top: 20px;
  z-index: 1000;
  display: flex;
  justify-content: center;
  gap: 20px;
  padding: 15px;
  background-color: #2b1a0c;
  width: 600px;
  max-width: 100%;
  margin: 0 auto;
}

.imagem-link-final {
  display: block;
  transition: transform 0.2s ease;
}

.imagem-link-final.botao-ampliado {
  transform: scale(1.1);
}
```

- `position: sticky` faz o menu acompanhar a rolagem.
- `top: 20px` mantém 20px de distância do topo.
- `z-index: 1000` coloca o menu na frente das imagens.
- `display: flex` organiza os links do menu.

- `justify-content: center` centraliza os links.
- `gap: 20px` separa os links.
- `transition` suaviza a transformação.
- `scale(1.1)` aumenta o elemento em 10%.

4. CSS responsivo

```
@media (max-width: 768px) {
  :root { padding: 20px; }
  #principal { max-width: 50%; }
  #nossa-historia { font-size: 40px; }
  .conteudo-paragrafo01 { flex-direction: column; }
  .conteudo-herculano { flex-direction: column; }
  #paragrafo01 { width: 100%; font-size: 18px; }
  #imagem-historia { width: 100%; }
  .acoes { grid-template-columns: 1fr; }
  .links-finais { grid-template-columns: 1fr; }
}
```

- `@media` aplica regras somente em telas de até 768px.
- O padding diminui para sobrar espaço no celular.
- O título reduz de 100px para 40px.
- Os blocos Flexbox passam de linha para coluna.
- O primeiro parágrafo passa a ocupar 100% da largura.
- O Grid de ações passa a uma coluna.
- Os links finais também passam a uma coluna.

5. JavaScript: interação

```
const botoes = document.querySelectorAll("nav a, .imagem-link-final");
```

`const` cria uma referência que não será substituída. `querySelectorAll` encontra todos os links do menu e os dois links de imagem.

```
botoes.forEach((botao) => {
  botao.addEventListener("mouseenter", () => {
    botao.classList.add("botao-ampliado");
  });

  botao.addEventListener("mouseleave", () => {
    botao.classList.remove("botao-ampliado");
  });
});
```

- `forEach` repete a mesma lógica para cada botão.
- `mouseenter` dispara quando o mouse entra.
- `classList.add` adiciona a classe que aumenta o elemento.
- `mouseleave` dispara quando o mouse sai.
- `classList.remove` remove a classe e devolve o tamanho normal.

Animação durante a rolagem

```
const elementosRevelados = document.querySelectorAll("h1, h2, p, img, .acoes");
```

Essa linha seleciona títulos, parágrafos, imagens e a grade de ações para que possam aparecer gradualmente.

```
const observador = new IntersectionObserver((entradas, observadorAtual) => {
```

```
entradas.forEach((entrada) => {
  if (entrada.isIntersecting) {
    entrada.target.classList.add("visible");
    observadorAtual.unobserve(entrada.target);
  }
});
}, { threshold: 0.15 });
```

- `IntersectionObserver` observa quando um elemento entra na área visível.
- `entradas` contém os elementos observados que mudaram de estado.
- `isIntersecting` informa se o elemento está visível.
- `classList.add("visible")` ativa a animação CSS.
- `unobserve` para de observar depois da primeira aparição.
- `threshold: 0.15` ativa quando aproximadamente 15% do elemento aparece.

```
elementosRevelados.forEach((elemento) => {
  elemento.classList.add("reveal");
  observador.observe(elemento);
});
```

- Adiciona a classe inicial `reveal`, que deixa o elemento transparente e deslocado.
- Registra cada elemento no observador.
- Quando ele aparece, a classe `visible` remove o deslocamento e a transparência.

6. *Resumo das funcionalidades*

- Menu sticky a 20px do topo e acima das imagens.
- Links do menu e imagens finais aumentam ao passar o mouse.
- Imagem do COPENE como fundo da seção Oportunidades, ampliada e quase apagada.
- Imagem ABPN e imagem COPENE 2026 na mesma linha em duas colunas iguais.
- As imagens finais têm a mesma largura e altura.
- Textos e imagens aparecem conforme a rolagem.
- Layout responsivo para celular.